

Análise de Complexidade

Felipe Eizo Hanada

Departamento de Ciência da Computação
Universidade Federal Fluminense
felipehanada@id.uff.br

Abril 2, 2026

Possíveis erros e imprecisões ao longo deste material!

Este material foi produzido por mim, atualmente aluno de graduação de Ciência da Computação por livre e espontânea vontade. Atente-se à erros e imprecisões que podem ocorrer ao longo deste texto – feedback será muito bem aceito.

1. Justificativa

- 1.1 Time Limit Exceeded – TLE
- 1.2 Estimando Tempo de Execução
- 1.3 Operações Elementares
- 1.4 Praticidade

2. Análise de Pior Caso

- 2.1 Notação Big-O
- 2.2 Fatores relevantes
- 2.3 Aplicação em Problemas
- 2.4 Algoritmos e Suas Complexidades

Time Limit Exceeded – TLE

FelipeHanada submissions

Status filter is used, click [here](#) to reset

#	When	Who	Problem	Lang	Verdict	Time	Memory
368975010	Mar/31/2026 07:50 UTC-3	FelipeHanada	C2 - A Simple GCD Problem (Hard Version)	C++20 (GCC 13-64)	Time limit exceeded on test 8	6000 ms	100 KB
368974270	Mar/31/2026 07:44 UTC-3	FelipeHanada	C2 - A Simple GCD Problem (Hard Version)	C++20 (GCC 13-64)	Time limit exceeded on test 8	6000 ms	100 KB
368974139	Mar/31/2026 07:43 UTC-3	FelipeHanada	C2 - A Simple GCD Problem (Hard Version)	C++20 (GCC 13-64)	Time limit exceeded on test 8	6000 ms	100 KB
368973956	Mar/31/2026 07:42 UTC-3	FelipeHanada	C2 - A Simple GCD Problem (Hard Version)	C++20 (GCC 13-64)	Time limit exceeded on test 8	6000 ms	100 KB
368205411	Mar/25/2026 22:04 UTC-3	FelipeHanada 	A - Collatz Conjecture	C++20 (GCC 13-64)	Time limit exceeded on test 1	1000 ms	0 KB
368092143	Mar/25/2026 00:00 UTC-3	FelipeHanada	D - From 1 to Infinity	C++20 (GCC 13-64)	Time limit exceeded on test 1	1500 ms	0 KB
366563307	Mar/13/2026 23:11 UTC-3	FelipeHanada	B - Buses	C++20 (GCC 13-64)	Time limit exceeded on test 18	1000 ms	19400 KB
366557443	Mar/13/2026 20:32 UTC-3	FelipeHanada	B - T-primes	C++20 (GCC 13-64)	Time limit exceeded on test 7	2000 ms	0 KB
366557391	Mar/13/2026 20:31 UTC-3	FelipeHanada	B - T-primes	C++20 (GCC 13-64)	Time limit exceeded on test 7	2000 ms	0 KB
365544131	Mar/06/2026 00:38 UTC-3	FelipeHanada	D - Divisibility Game	C++20 (GCC 13-64)	Time limit exceeded on test 2	2000 ms	15600 KB
365543313	Mar/06/2026 00:26 UTC-3	FelipeHanada 	D - Divisibility Game	C++20 (GCC 13-64)	Time limit exceeded on test 21	2000 ms	6000 KB
365543274	Mar/06/2026 00:26 UTC-3	FelipeHanada 	D - Divisibility Game	C++20 (GCC 13-64)	Time limit exceeded on test 2	2000 ms	2000 KB
365543223	Mar/06/2026	FelipeHanada	D - Divisibility Game	C++20 (GCC	Time limit exceeded on	2000 ms	2000 KB

→ Pay attention

Before contest

[April Fools Day Contest 2026](#)

14:14:05

[Register now »](#)

→ FelipeHanada



Rating: **1549**
Contribution: 0



FelipeHanada

- [Settings](#)
- [Blog](#)
- [Favourites](#)
- [Teams](#)
- [Submissions](#)
- [Problemsetting](#)
- [Groups](#)
- [Talks](#)
- [Contests](#)

→ Status filter

Contest:

Verdict:

Language:

Test:

Time Limit Exceeded – TLE


[Log in](#)
[Sign up](#)

[HOME](#)
[TOP](#)
[TODAYS](#)
[CONTESTS](#)
[GAM](#)
[PROBLEMS](#)
[GROUPS](#)
[RATING](#)
[ECON](#)
[API](#)
[CALENDAR](#)
[HELP](#)

[ALGORITHMS](#)
[METHODS](#)
[TOOLS](#)
[FAQ](#)
[ABOUT US](#)
[TEAM](#)
[SCHEDULE](#)
[DONATE](#)
[CONTACT](#)
[PRIVACY](#)
[POLICY](#)

[New contest](#)

Felipehenada submissions

Make a filter to avoid what you want

Felipehenada submissions

#	When	Who	Problem	Lang	Verdict	Time	Memory
336878019	Mar/15/2023 20:40:00 ^{UTC-3}	Felipehenada	C2 - A Simple DP Problem (Hard Version)	C++20 (GCC 13-64)	Time limit exceeded on test 8	6300 ms	100 KB
336878239	Mar/15/2023 20:44:00 ^{UTC-3}	Felipehenada	C2 - A Simple DP Problem (Hard Version)	C++20 (GCC 13-64)	Time limit exceeded on test 8	6300 ms	100 KB
336878439	Mar/15/2023 20:47:00 ^{UTC-3}	Felipehenada	C2 - A Simple DP Problem (Hard Version)	C++20 (GCC 13-64)	Time limit exceeded on test 8	6300 ms	100 KB
336878998	Mar/15/2023 20:50:00 ^{UTC-3}	Felipehenada	C2 - A Simple DP Problem (Hard Version)	C++20 (GCC 13-64)	Time limit exceeded on test 8	6300 ms	100 KB
336890411	Mar/15/2023 22:04:00 ^{UTC-3}	A - Grumpy Game	A - Grumpy Game	C++20 (GCC 13-64)	Time limit exceeded on test 1	1300 ms	0 KB
336892149	Mar/15/2023 22:09:00 ^{UTC-3}	Felipehenada	D - Reun To Safety	C++20 (GCC 13-64)	Time limit exceeded on test 1	1700 ms	0 KB
336893007	Mar/15/2023 22:15:00 ^{UTC-3}	Felipehenada	B - Bait	C++20 (GCC 13-64)	Time limit exceeded on test 18	1500 ms	16400 KB
336893483	Mar/15/2023 22:32:00 ^{UTC-3}	Felipehenada	8 - Trusses	C++20 (GCC 13-64)	Time limit exceeded on test 7	2300 ms	0 KB
336893789	Mar/15/2023 22:35:00 ^{UTC-3}	Felipehenada	8 - Trusses	C++20 (GCC 13-64)	Time limit exceeded on test 7	2300 ms	0 KB
336895131	Mar/15/2023 22:58:00 ^{UTC-3}	Felipehenada	D - Electricity Flow	C++20 (GCC 13-64)	Time limit exceeded on test 2	2300 ms	16800 KB
336895513	Mar/15/2023 22:59:00 ^{UTC-3}	Felipehenada	D - Electricity Flow	C++20 (GCC 13-64)	Time limit exceeded on test 21	2300 ms	6300 KB
336895274	Mar/15/2023 22:59:00 ^{UTC-3}	Felipehenada	D - Electricity Flow	C++20 (GCC 13-64)	Time limit exceeded on test 2	2300 ms	2300 KB
336895902	Mar/15/2023 23:01:00 ^{UTC-3}	Felipehenada	D - Electricity Flow	C++20 (GCC 13-64)	Time limit exceeded on test 2	2300 ms	2300 KB
336897173	Mar/15/2023 23:08:00 ^{UTC-3}	Felipehenada	D - Electricity Flow	C++20 (GCC 13-64)	Time limit exceeded on test 21	2300 ms	6300 KB
336897387	Mar/15/2023 23:08:00 ^{UTC-3}	Felipehenada	D - Electricity Flow	C++20 (GCC 13-64)	Time limit exceeded on test 21	2300 ms	61100 KB
336897018	Mar/15/2023 23:08:00 ^{UTC-3}	Felipehenada	D - Electricity Flow	C++20 (GCC 13-64)	Time limit exceeded on test 2	2300 ms	168100 KB
336897678	Mar/15/2023 23:09:00 ^{UTC-3}	Felipehenada	D - Electricity Flow	C++20 (GCC 13-64)	Time limit exceeded on test 8	2300 ms	168100 KB
336898068	Mar/15/2023 23:09:00 ^{UTC-3}	Felipehenada	D - Electricity Flow	C++20 (GCC 13-64)	Time limit exceeded on test 8	2300 ms	2300 KB
336898511	Mar/15/2023 23:08:00 ^{UTC-3}	Felipehenada	D - Electricity Flow	C++20 (GCC 13-64)	Time limit exceeded on test 2	2300 ms	2300 KB
336898208	Mar/15/2023 23:33:00 ^{UTC-3}	Felipehenada	D - Electricity Flow	C++20 (GCC 13-64)	Time limit exceeded on test 2	2300 ms	106600 KB
336898874	Mar/15/2023 23:26:00 ^{UTC-3}	Felipehenada	D - Electricity Flow	C++20 (GCC 13-64)	Time limit exceeded on test 2	2300 ms	97100 KB
336897378	Mar/15/2023 23:08:00 ^{UTC-3}	Felipehenada	D - Grumpy Game	C++20 (GCC 13-64)	Time limit exceeded on test 1	1300 ms	0 KB
336898049	Mar/15/2023 11:48:00 ^{UTC-3}	Felipehenada	8 - Reunion and Election for Reunion	C++20 (GCC 13-64)	Time limit exceeded on test 1	1300 ms	0 KB
336898468	Mar/15/2023 11:42:00 ^{UTC-3}	Felipehenada	8 - Reunion and Election for Reunion	C++20 (GCC 13-64)	Time limit exceeded on test 1	1300 ms	0 KB
336898904	Mar/15/2023 11:42:00 ^{UTC-3}	Felipehenada	8 - Reunion and Election for Reunion	C++20 (GCC 13-64)	Time limit exceeded on test 1	1300 ms	0 KB

Pay attention

Before contest
April Fools Day Contest 2023
Join the contest
[Register now >](#)

Felipehen

Estimando Tempo de Execução

Codificar soluções custa tempo, como podemos saber se um algoritmo dará **TLE** antes de implementá-los ou submetê-los*?

Submissões incorretas*

Tome cuidado com submissões incorretas! Em *contests* do formato ICPC, cada submissão incorreta (em uma questão posteriormente resolvida) conta para a sua penalidade; e em outros *contests* podem configurar outros tipos de penalidade, como débitos na sua pontuação final.

Estimando Tempo de Execução

Operações realizadas por computadores podem tomar tempos diferentes de execução dependendo de muitos fatores como:

- implementação do hardware;
- SO utilizado;
- estado da memória;
- tamanho da entrada;
- seed de geradores de números aleatórios;
- (raios cósmicos?);
- ambiente de execução;
- poder computacional da máquina;
- disponibilidade de recursos;
- etc.

Estimando Tempo de Execução

Então, o que fazer? Vamos utilizar uma métrica razoável que abstrai a ideia de "tempo de execução" para o algoritmo, ao invés do ambiente no qual ele é executado.

Assim, a pergunta vira: "Quantos *passos* meu algoritmo precisa dar para terminar?", ao invés de em quanto tempo ele irá terminar.

Operações Elementares

Um "passo" possui definição bastante informal para esse tipo de análise. Consideramos um passo como uma das seguintes operações elementares comumente implementadas de maneira eficiente e com tempo de execução bem comportado em computadores:

- Leitura/Escrita de espaços de um único espaço de memória;
- Avaliação de operações aritméticas;
- Avaliação de operações lógicas;
- Controle de fluxo (chamadas, retornos, if-else, loops);

Operações Elementares

```
1: procedimento SOMADEPA( $n, a_0, r$ )  
2:    $res \leftarrow 0$   
3:   para  $i = 1..n$  faça  
4:      $res \leftarrow res + a_0 + r(i - 1)$   
5:   fim para  
6:   retorne  $res$   
7: fim procedimento
```

Quantos *passos* este algoritmo leva para terminar?

Operações Elementares

```
1: procedimento SOMADEPA( $n, a_0, r$ )  
2:    $res \leftarrow 0$   
3:   para  $i = 1..n$  faça  
4:      $res \leftarrow res + a_0 + r(i - 1)$   
5:   fim para  
6:   retorne  $res$   
7: fim procedimento
```

Quantos *passos* este algoritmo leva para terminar?

$$t(n) = 1 + n(1 + 1 + 1 + 1 + 1 + 1) + 1 = 6n + 2 \text{ passos}$$

Uma boa estimativa para a quantidade de operações por segundo que um juiz realiza é da ordem de $\approx 10^8$.

Mas vale mesmo a pena contar cada operação realizada pelo seu algoritmo? Dica: não.

Praticidade

Uma boa estimativa para a quantidade de operações por segundo que um juiz realiza é da ordem de $\approx 10^8$.

Mas vale mesmo a pena contar cada operação realizada pelo seu algoritmo? Dica: não.

Computadores modernos são tão rápidos, que não precisamos ser tão pedantes ao ponto de contar com precisão a quantidade de operações realizadas. As entradas possuem ordens de grandeza que tornam insignificantes alguns termos e fazem com que poucos termos sejam suficientes para descrever o comportamento para entradas suficientemente grandes.

Exemplo anterior

Tome o exemplo anterior, onde $t(n) = 6n + 2$, quanto mais aumentamos o valor de n , o fator constante 2 contribui cada vez menos para nossa estimativa. E se você aumentar ainda mais o valor de n , perceberá que $6n$ ou apenas n , é suficiente para você tirar suas conclusões. (tente plottar no geogebra ou desmos)

Notação Big-O

Definição 1

Dada duas funções reais f e g , dizemos que f é $O(h)$ ou $f = O(h)$ se existe um real $c > 0$ e um inteiro n_0 tais que:

$$f(n) \leq cg(n) : \text{para todo } n > n_0$$

Soma de PA

Nosso algoritmo de soma de PA é $O(n)$, vamos provar:

Basta tomar $c = 7$ e $n_0 = 2$, e vemos que a inequação

$$f(n) \leq cg(n) \iff 6n + 2 \geq 7n \iff n \geq 2 = n_0 \text{ é satisfeita.}$$

Você pode provar que f também é $O(n^2)$, $O(n!)$, $O(n^3)$ e até $O(n^n)$, mas essas estimativas são "largas" demais.

Fatores Mais e Menos Relevantes

A grande vantagem da notação O é a simplicidade. Podemos calcular de cabeça para a maioria dos algoritmos que vamos encontrar e ela fornece estimativas suficientemente boas para o nosso planejamento de solução.

As seguintes propriedades são verdades (tente pensar como você verificaria elas):

- $O(kf) = O(f)$; (constantes multiplicativas)
- $O(f + g) \geq O(f), O(g)$; (algoritmo em etapas)
- $O(f)O(g) = O(fg)$; (algoritmos aninhados)

Nota: isso nem sempre gera complexidades justas, veja *análise amortizada*.

- $O(\log_a n) = O(\log_b n)$ (indiferença de base de log)
- $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

n	Possible complexities
$n \leq 10$	$\mathcal{O}(n!), \mathcal{O}(n^7), \mathcal{O}(n^6)$
$n \leq 20$	$\mathcal{O}(2^n \cdot n), \mathcal{O}(n^5)$
$n \leq 80$	$\mathcal{O}(n^4)$
$n \leq 400$	$\mathcal{O}(n^3)$
$n \leq 7500$	$\mathcal{O}(n^2)$
$n \leq 7 \cdot 10^4$	$\mathcal{O}(n\sqrt{n})$
$n \leq 5 \cdot 10^5$	$\mathcal{O}(n \log n)$
$n \leq 5 \cdot 10^6$	$\mathcal{O}(n)$
$n \leq 10^{18}$	$\mathcal{O}(\log^2 n), \mathcal{O}(\log n), \mathcal{O}(1)$

Figura: Tabela retirada da seção de Time Complexity (Bronze) do USACO GUIDE.

Nota: essa tabela não garante que seu algoritmo irá passar mesmo se ele tiver a complexidade mostrada, se trata de uma heurística – refira ao material original para mais detalhes.

Vamos aplicar em soluções reais de problemas!

1. <https://codeforces.com/contest/2210/submission/368693442>
2. <https://codeforces.com/contest/2210/submission/368732251>
3. <https://codeforces.com/contest/2210/submission/368722526>
4. <https://codeforces.com/contest/2210/submission/368975590>
5. <https://atcoder.jp/contests/abc451/submissions/74593363> (Análise Amortizada)

Vamos aplicar em soluções reais de problemas!

1. <https://codeforces.com/contest/2210/submission/368693442>
Complexidade: $O(n)$
2. <https://codeforces.com/contest/2210/submission/368732251>
Complexidade: $O(n \log n)$
3. <https://codeforces.com/contest/2210/submission/368722526>
Complexidade: $O(n \log(\max A))$
4. <https://codeforces.com/contest/2210/submission/368975590>
Complexidade: $O(nL^2 \log(\max A))$
5. <https://atcoder.jp/contests/abc451/submissions/74593363>
Complexidade: $O(n \log n)$

Casos mais difíceis

Os itens 3 e 4 são casos mais difíceis e só servem para mostrar o quão longe você pode levar essa análise, não se preocupe se não conseguir entender imediatamente.

Algoritmos e Suas Complexidades

- Inserção e Remoção em RBTrees (sets e maps na STL) – $O(\log n)$
- Algoritmo de Kadane – $O(n)$
- Crivo de Erathostenes – $O(n \log^2 n)$
Curiosidade: procure sobre o Crivo Linear de Erathostenes.
- Busca Binária – $O(\log n)$
- Algoritmo de Euclides, GCD e LCM – $O(\log(\min\{a, b\}))$
- LIS e LDS – $O(n \log n)$
- BFS/DFS – $O(n + m)$
Em árvores, esse algoritmo roda em $O(n)$
- Algoritmo de Kruskal e Prim – $O(n \log n)$
- Algoritmo de FloydWarshall – $O(n^3)$
- ...

References I



Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022).

Introduction to Algorithms.

MIT Press, 4th edition.

Capítulo 3: Characterizing Functions (Asymptotic Notation).



Cunha, L. F. I. (2022).

Aula 2 - eficiência computação (parte ii): Notação o, omega, theta, algoritmos Ótimos.

YouTube.

Vídeo aula disponível no canal oficial do autor.



Laaksonen, A. (2018).

Competitive Programmer's Handbook.

Capítulo 2: Time complexity.

References II



USACO Guide.

Time complexity.

<https://usaco.guide/bronze/time-comp>.

Acessado em: 31 de março de 2026.